

Quick review from k-NN

Scenario: You're building a model to predict whether hospital patients will be readmitted within 30 days. You have:

- 10,000 patient records
- 50 features (demographics, diagnoses, medications, lab values)
- 8% of patients are actually readmitted

Discuss with your group:

- How would you split this data for model selection and evaluation?
- Would you use cross-validation? How many folds?
- Which metric(s) should you prioritize? Why?
- Would k-NN be a good choice here? What concerns do you have?

Intro to k-means clustering

From supervised to unsupervised learning

In the last few lectures, we learned supervised learning with k-NN:

Given: data with labels $\{(x_i, y_i)\}$

Goal: predict labels for new data

What if we don't have labels? What can we still learn from data?

Unsupervised learning: discover structure in data without labels.

Today's goal: clustering — grouping similar data points together.

Why Cluster?

When might you want to group data without knowing the groups in advance?

Real-world applications:

Customer segmentation: group customers by purchasing behavior

Document organization: group news articles by topic

Image compression: group similar colors together

Biology: group genes by expression patterns

Anomaly detection: find points that don't fit any group

Similarity and dissimilarity

To cluster, we need to measure how *similar* or *dissimilar* two data points are.

We work with many types of data:

- User records
- Images
- Text documents
- DNA sequences
- Time series
- ...

Key question: How do we define “similar” mathematically?

Dissimilarity as a distance

A *dissimilarity function* $d(x, y)$ returns a large value when x and y are different.

The most common type is a *metric* (also called a *distance function*).

A function $d(x, y)$ is a *metric* if it satisfies:

Property	Meaning
$d(x, y) = 0 \Leftrightarrow x = y$	Zero distance means identical
$d(x, y) \geq 0$	Distances are non-negative
$d(x, y) = d(y, x)$	Symmetric
$d(x, z) \leq d(x, y) + d(y, z)$	Triangle inequality

The triangle inequality

The *triangle inequality* says: the direct path is never longer than going through a third point.



$$d(x, z) \leq d(x, y) + d(y, z)$$

Euclidean distance satisfies all metric properties — it's our go-to dissimilarity measure.

Is "number of different characters" a metric for strings? (e.g., $d(\text{"cat"}, \text{"car"}) = 1$) why/why not?

Cosine similarity

Sometimes we care about *direction*, not magnitude.

Example: Comparing documents by word frequency.

Document 1 (short):

$$\mathbf{x} = \begin{bmatrix} 3 \\ 7 \\ 4 \end{bmatrix} \leftarrow \begin{bmatrix} \text{dog} \\ \text{cat} \\ \text{house} \end{bmatrix}$$

Document 2 (long):

$$\mathbf{y} = \begin{bmatrix} 30 \\ 70 \\ 40 \end{bmatrix}$$

Same topics, just 10x longer!

Euclidean distance says these are very different. But they're about the same topics!

$$\mathbf{x}^T \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos(\theta)$$

Recall: The dot product relates to the angle between vectors:

$$\cos(\theta_{\mathbf{x},\mathbf{y}}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

Rearranging gives us cosine similarity:

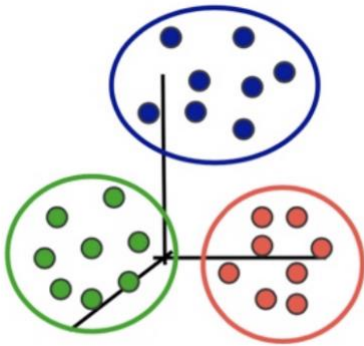
- *cos* = 1: vectors point the same direction (most similar)
- *cos* = 0: vectors are perpendicular (unrelated)
- *cos* = -1: vectors point opposite directions

To convert to *dissimilarity*: use $1 - \cos(\theta)$

What is clustering?

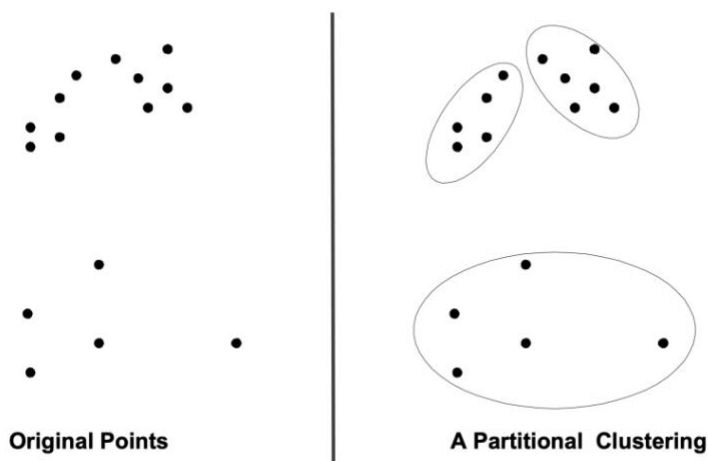
A **clustering** is a grouping of data where:

- Points **within** a group are similar
- Points in **different** groups are dissimilar



More formally: **minimize** intra-cluster distances, **maximize** inter-cluster distances.

We focus on **partitional clustering**: each point belongs to exactly one cluster, and every point is assigned.



Application: Image compression

Each pixel has a color (R, G, B). With 8 bits per channel, that's $2^{24} \approx 16$ million possible colors.

Idea: Cluster similar colors together, replace each pixel with its cluster's "average" color.

Fewer colors = fewer bits needed = compression!

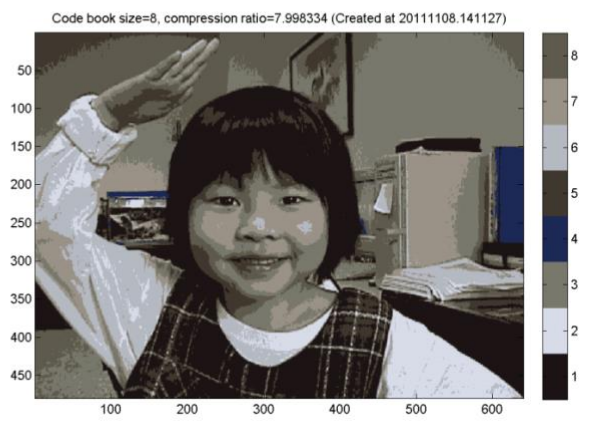


Image compression results

16 colors (4 bits)



8 colors (3 bits)



4 colors (2 bits)



2 colors (1 bit)



The Centroid

To define k -means, we need the *centroid* (also called *center* or *mean*).

Given vectors $\mathbf{x}_1, \dots, \mathbf{x}_n$, the centroid is their *average*:

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$

The centroid is the “center of mass” of the points.

Find the centroid of $\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 6 \\ 2 \end{bmatrix}$

The k – means problem

Given:

- Data points $\mathbf{x}_1, \dots, \mathbf{x}_n$ in $\mathbb{R}^n$
- Number of clusters k

Find:

k centroids $\mathbf{c}_1, \dots, \mathbf{c}_k$ that minimize the *cost function*:

$$\text{Cost} = \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{c}_{j(i)}\|^2$$

where $j(i)$ is the index of the centroid closest to $\mathbf{x}_i$.

In other words: *minimize the total squared distance from each point to its nearest center.*

The k – means algorithm

How hard is this?

Some special cases are easy:

- $k = 1$: centroid is just the mean of all points
- $k = n$: each point is its own cluster

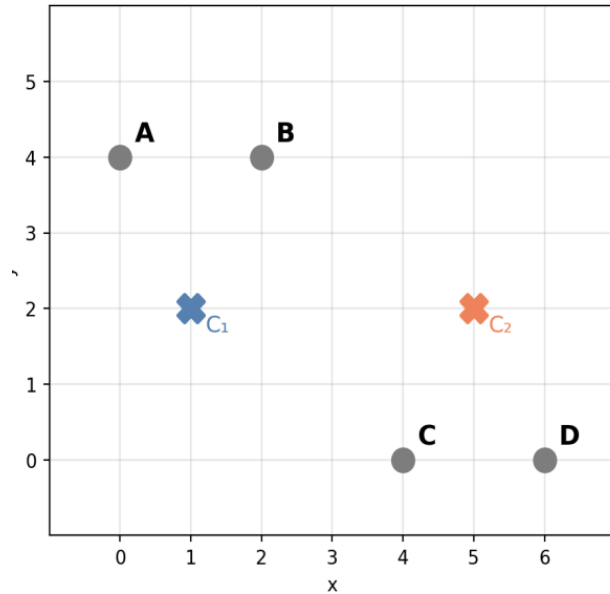
But in general? The problem is *NP-hard*— no efficient exact algorithm exists.

The good news: there's a simple algorithm that works well in practice. It's one of the top 10 algorithms in data mining!

The algorithm

- *Initialize*: Pick k starting centers (randomly or strategically)
- *Assign*: Put each point in the cluster of its nearest center
- *Update*: Recompute each center as the centroid of its cluster
- *Repeat* steps 2-3 until nothing changes

Practice: k – means by hand



Data points:

$A = (0, 4), B = (2, 4), C = (4, 0), D = (6, 0)$

Initial centers:

$C_1 = (1, 2), C_2 = (5, 2)$

Work through one iteration:

- Assign each point to its nearest center
- Compute new center positions
- Sketch the new centers on the plot

The clustering workflow

A typical clustering project follows these steps:

- *Prepare data*: clean, normalize, select features
- *Choose k* : how many clusters? (more on this next lecture)
- *Run algorithm*: fit k-means to the data
- *Visualize*: do the clusters make sense?
- *Interpret*: what does each cluster represent?
- *Validate*: are the clusters meaningful?

Our Dataset: 30 dimensions

Visualizing the dataset

Suppose we have a dataset that contains 100 data points, each with 30 features.

```
1 X, y_true = sk_data.make_blobs(n_samples=100, centers=3, n_features=30,  
2                               center_box=(-10.0, 10.0), random_state=0)  
3 print(f"Shape: {X.shape}")  
4 print(f"100 points, each with 30 features")
```

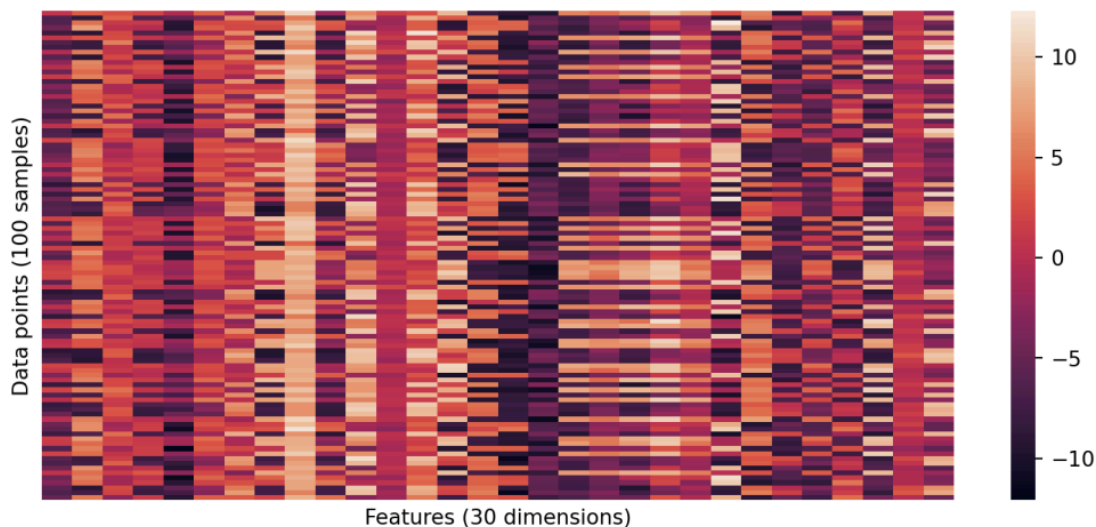
Shape: (100, 30)

100 points, each with 30 features

How do you visualize 30 dimensions? You can't plot it directly!

One option: *Heatmap!*

Each row is a point, each column is a feature.



Do you see any clusters here? What makes it so hard to see?

Our Dataset: 30 dimensions

Visualizing the dataset

Pairwise distances

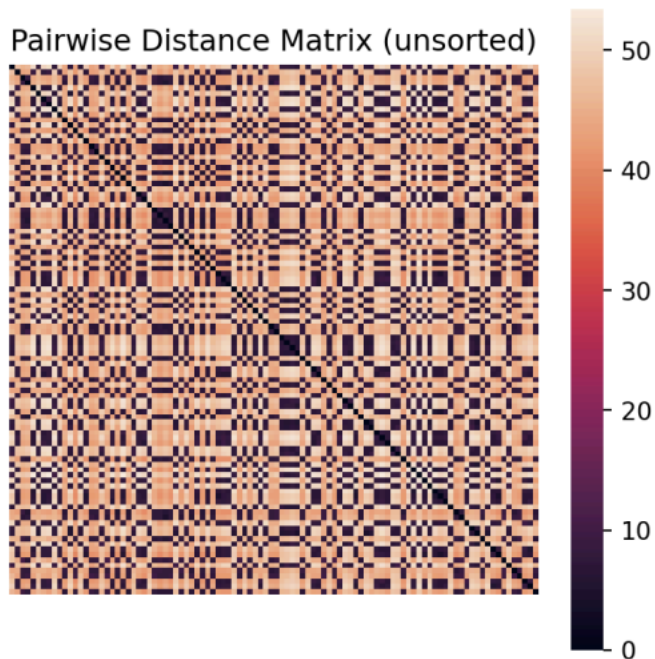
Another view: compute the distance between every pair of points.

```
1 euclidean_dists = metrics.euclidean_distances(X)
2 print(f"Distance matrix shape: {euclidean_dists.shape}")
3 print(f"Distance from point 0 to point 1: {euclidean_dists[0, 1]:.2f}")
```

Distance matrix shape: (100, 100)

Distance from point 0 to point 1: 47.74

This gives us a 100×100 matrix. Entry (i, j) = distance between points i and j .



Our Dataset: 30 dimensions

The visualization problem

Heatmaps show raw values or distances, but hard to interpret.

Can we find 2D coordinates that ***preserve*** the distances between points?

This is exactly what dimensionality-reduction tools like ***Multidimensional Scaling (MDS), t-SNE, and UMAP do!***

Our Dataset: 30 dimensions

Multidimensional Scaling (MDS)

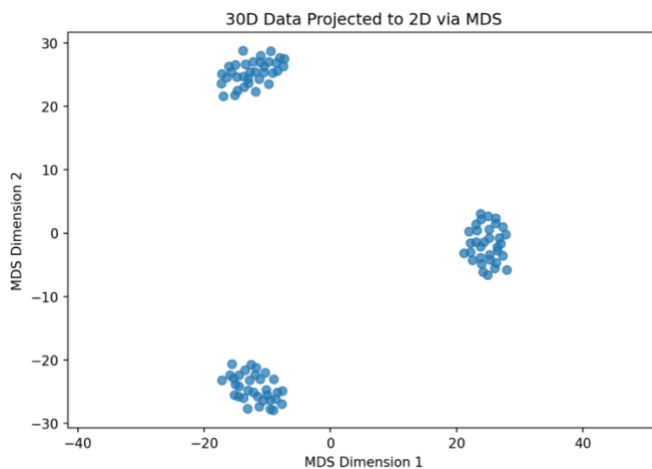
Goal: Find 2D coordinates where distances match the original high dimensional distances.

How it works:

- Start with random 2D positions
- Compare distances in 2D to true distances in 30D
- Adjust positions to reduce the mismatch
- Repeat until distances are as close as possible

In practice, t-SNE and UMAP are more common, but the algorithms are more complex.

```
1 mds = sklearn.manifold.MDS(n_components=2, max_iter=3000, eps=1e-9
2                             random_state=0, dissimilarity="precomp
3 pos = mds.fit_transform(euclidean_dists)
```



Running *k*-means

scikit-learn provides `KMeans` in `sklearn.cluster`:

```
1 kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=0)
2 kmeans.fit(X);
```

Key parameters:

- *n_clusters*: the *k* in *k*-means
- *init='k-means++'*: smart initialization (more next lecture)
- *n_init*: run algorithm multiple times, keep best result

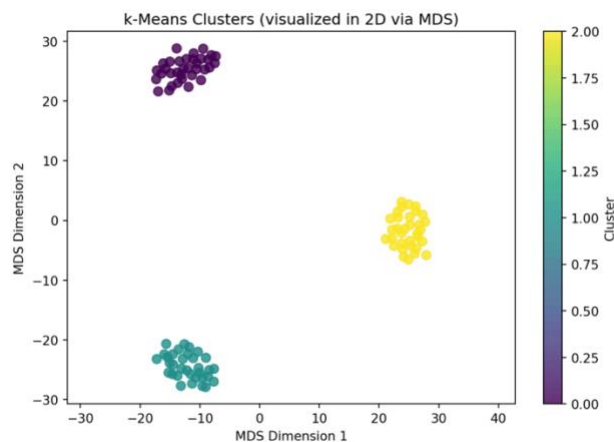
Examining the results

```
1 # Cluster assignments for each point
2 labels = kmeans.labels_
3 print(f"Labels: {labels}")
4
5 # Cluster centers (30-dimensional vectors)
6 print(f"\nCentroid shapes: {kmeans.cluster_centers_.shape}")
7
8 # Total within-cluster sum of squares (the cost function)
9 print(f"\nInertia (cost): {kmeans.inertia_:.1f}")
```

```
Labels: [0 1 2 2 1 0 1 1 0 1 2 0 2 0 1 2 2 0 2 1 2 1 2 0 1 1 0 2 2 2 0 2 0 2 1 2
 1 2 1 1 1 0 2 0 2 0 1 0 2 2 0 0 0 0 2 0 1 1 2 0 2 1 0 0 1 2 0 0 1 1 1 0 0
 2 0 1 0 1 0 1 1 1 0 2 2 1 0 0 2 0 1 1 2 0 2 2 1 1 2]
```

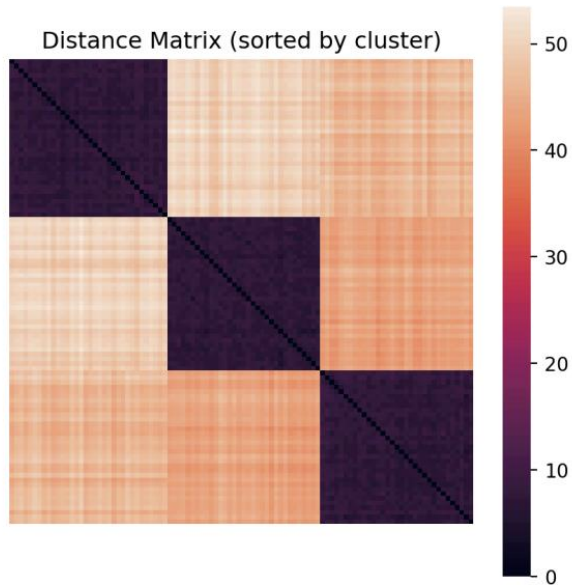
Centroid shapes: (3, 30)

Inertia (cost): 2733.8



Sorted distance matrix

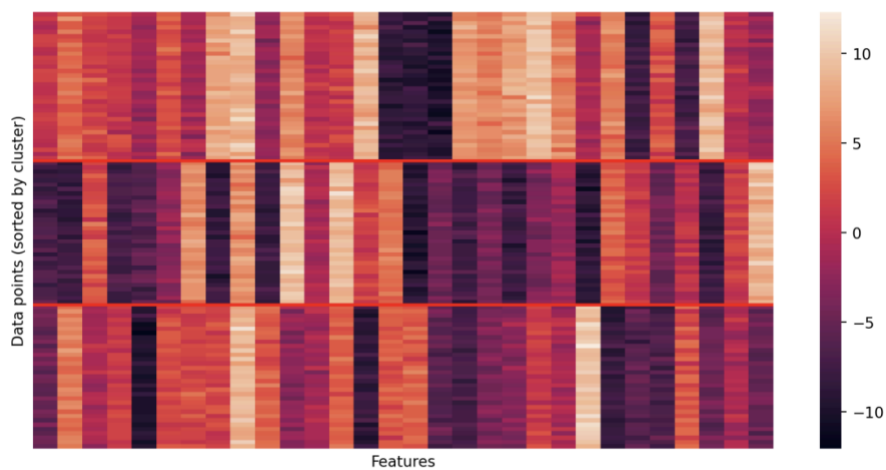
Let's reorder points by cluster and look at the distance matrix again:



Dark blocks on diagonal = points in same cluster are close. Success!

Sorted heatmap

Same idea for the raw data — sort rows by cluster:



Now we can see that clusters have different feature patterns!

Interpreting clusters

We found 3 clusters — but what do they mean?

For *synthetic data*: we know the “true” labels, so we can check.

For *real data*: we need to examine the clusters to understand them:

- Look at cluster centroids (what’s the “average” member like?)
- Look at representative examples from each cluster
- Compare feature distributions across clusters
- Use domain knowledge to assign meaningful labels

Our synthetic data has true labels `y_true`. Let’s compare:

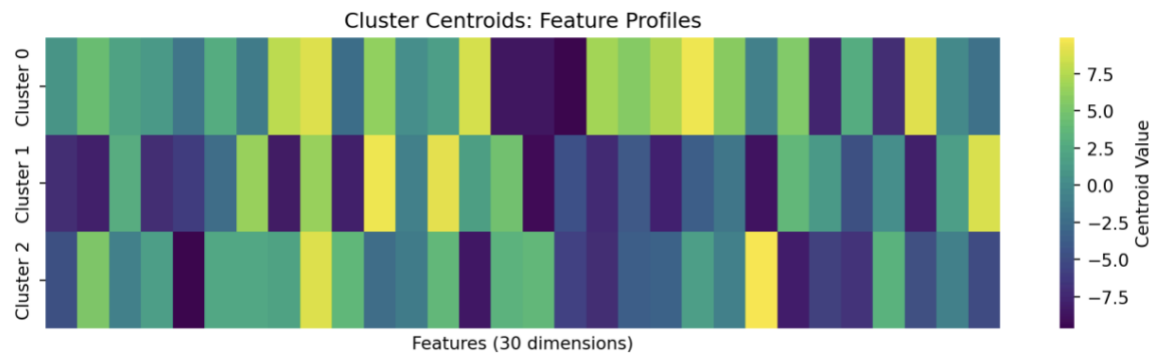
```
1 # Create a cross-tabulation
2 import pandas as pd
3 comparison = pd.crosstab(y_true, labels,
4                           rownames=['True Label'],
5                           colnames=['k-Means Label'])
6 print(comparison)
```

k-Means Label	0	1	2
True Label			
0	34	0	0
1	0	0	33
2	0	33	0

Why are the labels "permuted"? Does k-means cluster 0 have to correspond to true label 0? Why or why not?

Cluster Profiles

What makes each cluster different? Look at the centroids:



Each row shows a cluster's “signature” — distinct patterns across features.

Practice: Interpret these clusters:

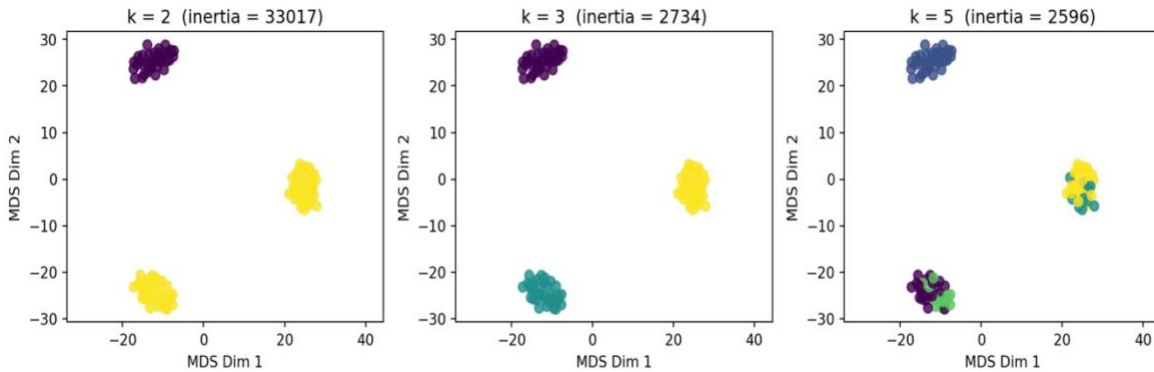
A retailer clustered their customers and got these centroids:

	Age	Income (\$K)	Monthly Spend (\$)	Visits/Month
Cluster 0	42	55	150	8
Cluster 1	28	95	420	3
Cluster 2	58	140	180	2

- Give each cluster a descriptive name
- What marketing strategy might work for each?

What if we pick the wrong k ?

We used $k = 3$ because we generated 3 clusters. But what if we didn't know?



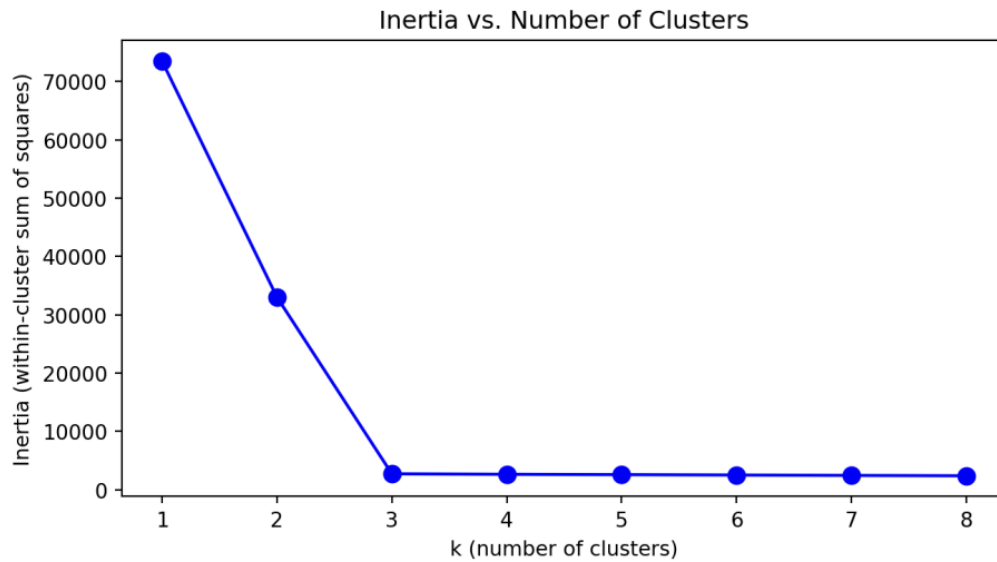
Look at the plots:

- $k = 2$: Two real clusters got merged. Which ones?
- $k = 5$: Some real clusters got split. Does this seem useful?
- The inertia (cost) decreases as k goes up. Why?

Lower inertia doesn't always mean a better model — we need smarter ways to choose k !

Preview: How inertia changes with k

Predict: Where is the "best" k on this plot? How would you decide?



Workflow Summary

Clustering Workflow:

- *Visualize:* Use MDS or similar to see if clusters exist
- *Cluster:* Run k-means with chosen k
- *Validate:* Check distance matrix, compare to ground truth if available
- *Interpret:* Examine centroids and cluster members
- *Iterate:* Try different k values, compare results

